

Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration

Prüfungsfach:	Betriebliche IT-Prozesse	
Fortbildungsprüfung:	Certified IT Business Manager (IHK)	
Prüfling:	Daniel Massa, Prüflingsnummer 615951	
Projektbetrieb:	Verein zur Förderung der Berufsbildung e.V., Kurfürstenstraße 6, 71636 Ludwigsburg	Erklärung:
Wohnanschrift:	Hackstraße 41, 70190 Stuttgart	
Abgabedatum:	01.11.2026	
Betreuer im Betrieb:	Carsten Vordermeier	
IHK-Betreuer:	Frau Dr. Sabine Wagner, IHK Stuttgart	

Die Projektarbeit wurde selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt. Stuttgart, den 01.11.2026 _____

Daniel Massa

Sperrvermerk

Die vorliegende Projektarbeit ist vertraulich zu behandeln. Eine Weitergabe an Dritte ist ohne Zustimmung des Auftraggebers nicht gestattet.

Contents

Sperrvermerk

Abbildungsverzeichnis iv

Tabellenverzeichnis v

Abkürzungsverzeichnis vi

1 Initiierung	1
1.1 Projektumfeld und Ausgangssituation	1
Projektziel	1
Projektbegründung	1
Projektschnittstellen	1
Projektabgrenzung	1
1.2 Zieldefinition	1
Projektbegründung	2
Projektschnittstellen	2
Projektabgrenzung	2
1.3 Projektrahmen	2
Projektschnittstellen	2
Projektabgrenzung	3
1.4 Abgrenzung der Projekteinhalte	3
Projektabgrenzung	3
1.5 Projekialternativen	3
Nutzwertanalyse	4
Anwendungsfälle (Use Cases)	4
Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)	4
Lastenheft/Fachkonzept	4
1.6 Lastenheft / Anforderungen	5
1.7 Projektteam / Rollen	5
1.8 Projektorganisation	5
1.9 Projektauftrag	5
2 Projektplanung	6
2.1 Kick-Off / Projektstart	6
Abweichungen vom Projektantrag	6
Ressourcenplanung	6
Entwicklungsprozess	6
Stakeholderanalyse	6
2.2 Stakeholderanalyse	7
Projektabgrenzung	7
2.3 Kommunikationsplan	7
2.4 Ist-Analyse	8
Wirtschaftlichkeitsanalyse	8
Nutzwertanalyse	9
Anwendungsfälle (Use Cases)	9
Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)	9
Lastenheft/Fachkonzept	9
2.5 Anforderungsanalyse	9
Anwendungsfälle (Use Cases)	9
Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)	10
Lastenheft/Fachkonzept	10

2.6 Machbarkeitsanalyse	10
Nutzwertanalyse	10
Anwendungsfälle (Use Cases)	11
Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)	11
Lastenheft/Fachkonzept	11
Nutzwertanalyse	11
Anwendungsfälle (Use Cases)	11
Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)	12
Lastenheft/Fachkonzept	12
2.7 Risikoanalyse	12
2.8 Projektstrukturplan	12
2.9 Vorgehensmodell	13
Stakeholderanalyse	13
2.10 Meilensteine	13
2.11 Arbeitspakete	14
2.12 Ablauf- und Terminplanung	14
2.13 Kostenplanung	14
Entwicklungsprozess	14
Stakeholderanalyse	14
2.14 Qualitätsmanagement	15
2.15 Projektcontrolling	15
3 Konzeptionierung	15
3.1 Technologieauswahl	15
Architekturdesign	16
Benutzeroberfläche (Mockups)	16
Datenmodell (Entity-Relationship)	16
Geschäftslogik	16
Qualitätssicherung	17
Pflichtenheft / DSGVO-Konzept	17
3.2 Berechtigungskonzept / RBAC	17
Qualitätssicherung	17
Pflichtenheft / DSGVO-Konzept	17
3.3 Architekturentscheidungen	18
Benutzeroberfläche (Mockups)	18
Datenmodell (Entity-Relationship)	18
Geschäftslogik	18
Qualitätssicherung	19
Pflichtenheft / DSGVO-Konzept	19
3.4 GitHub-/CI-CD-Konzept	19
3.5 Schnittstellen / Datenflüsse	20
Geschäftslogik	20
Qualitätssicherung	20
Pflichtenheft / DSGVO-Konzept	20
3.6 Testkonzept	21
Pflichtenheft / DSGVO-Konzept	21
4 Durchführung und Steuerung	22
4.1 Umsetzung GitHub-Repository	22
Benutzeroberfläche	22
Geschäftslogik & GitHub Integration	22
4.2 Umsetzung CI/CD-Workflow	23
Geschäftslogik & GitHub Integration	23

4.3 Umsetzung Policy-/Audit-Komponenten	24
4.4 Testszenarien durchführen und auswerten	25
4.5 Kontinuierliche Abstimmung	25
4.6 Kostenkontrolle	25
4.7 Terminkontrolle	25
5 Abschluss	25
5.1 Projektabnahme	25
5.2 Lessons Learned	25
Ausblick	26
5.3 Projektabschlussbericht	26
5.4 Nachkalkulation	26
5.5 Nacharbeiten / Betrieb	26
5.6 Persönliches Fazit	27
6 Anhang	27
6.1 RACI-Matrix	27
6.2 Stakeholderanalyse	27
6.3 Kommunikationsplan	28
6.4 Anforderungen / Testfälle	28
6.5 Risikoanalyse	30
6.6 Projektstrukturplan	30
6.7 Architektur- und Ablaufdiagramme	32
6.8 Screenshots / Systemnachweise	32
6.9 Audit-/CI-CD-Nachweise	36
6.10 ML/KI-Erweiterung als experimenteller Folgeausbau	36
Literaturverzeichnis	36
Normen & Standards	36
Fachliteratur	37
Projektmanagement	37
IT-Security & Compliance	37
Weitere Quellen	37
Eidesstattliche Versicherung	37

Abbildungsverzeichnis

- Abbildung 1: Projektstrukturplan (PSP)
- Abbildung 2: Stakeholder-Matrix
- Abbildung 3: Risikomatrix
- Abbildung 4: Projektzeitplan (Gantt-Diagramm)
- Abbildung 5: Kostenverlauf Soll/Ist
- Abbildung 6: Meilenstein-Trend-Analyse
- Abbildung 7: Kommunikationsstruktur
- Abbildung 8: Zielarchitektur Zero Trust
- Abbildung 9: Sicherheitszonen- und Vertrauensmodell
- Abbildung 10: Datenfluss und Kontrollpunkte
- Abbildung 11: CI/CD-Pipeline mit Security Gates
- Abbildung 12: Test- und Abnahmeprozess
- Abbildung 13: Betriebsprozess nach Go-Live
- Abbildung 14: Monitoring- und KPI-Dashboard
- Abbildung 15: RACI-Matrix
- Abbildung 16: Self-Service: Rollenbeantragung
- Abbildung 17: Genehmigungs-Dashboard
- Abbildung 18: GitHub Actions Workflow (YAML)
- Abbildung 19: Actions Run – Testnachweis
- Abbildung 20: Secret-Scanning (Security-Test)
- Abbildung 21: RBAC-Matrix / Teamstruktur
- Abbildung 22: Audit-Log-Auszug (Revisionssicherheit)
- Abbildung 23: GitHub Repository Übersicht
- Abbildung 24: Terminal – Testausgabe (pytest)

Tabellenverzeichnis

Tabelle 1: Zeitplanung 70 Stunden
Tabelle 2: Kostenplanung
Tabelle 3: Stakeholdermatrix
Tabelle 4: Risikomatrix (1-5 Skala)
Tabelle 5: Nutzwertanalyse (1-5)
Tabelle 6: Make-or-Buy
Tabelle 7: Anforderungsmatrix
Tabelle 8: Testfallmatrix
Tabelle 9: Soll-Ist-Vergleich
Tabelle 10: Kommunikationsmatrix
Tabelle 11: RACI-Matrix
Tabelle 12: KPI-Matrix

Abkürzungsverzeichnis

Abkürzung	Bedeutung
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DSGVO	Datenschutz-Grundverordnung
ERD / ERM	Entity Relationship Diagram / Model
GitHub	GitHub Enterprise / GitHub.com
IAM	Identity and Access Management
KPI	Key Performance Indicator
MTA	Meilensteintrendanalyse
PSP	Projektstrukturplan
RBAC	Role-Based Access Control
RACI	Responsible, Accountable, Consulted, Informed
REST	Representational State Transfer
SIEM	Security Information and Event Management
SQL	Structured Query Language
SSO	Single Sign-On
TDD	Test-Driven Development
UI/UX	User Interface / User Experience
YAML	YAML Ain't Markup Language
Zero Trust	Zero-Trust-Sicherheitsmodell (Never Trust, Always Verify)

1 Initiierung

1.1 Projektumfeld und Ausgangssituation

Der **Verein zur Förderung der Berufsbildung (VFB)** bietet als gemeinnütziger, regionaler Bildungsträger zahlreiche IHK- und IT-Qualifizierungen an. Mit rund 50 Beschäftigten und hybriden Lernplattformen ist die Digitalisierung zentraler Unternehmensfokus. Ziel ist es, mit effizienten, automatisierten Workflows unter Gewährleistung maximaler IT- und Datenschutzvorgaben innovative Vorreiterrolle einzunehmen.

Projektziel

Bis **01.11.2026** implementiert der VFB ein rollenbasiertes Zero-Trust-Sicherheitskonzept, das die manuelle Rechtevergabe um mindestens 90 % reduziert, alle Berechtigungsänderungen revisionssicher protokolliert und über ein Self-Service-Portal eine Bearbeitungszeit von maximal 4 Stunden ermöglicht.

Messbare Erfolgskriterien: | Kriterium | Zielwert | Bezug | |—|—|—| | Bearbeitungszeit pro Anfrage | maximal 4 Stunden | Self-Service-Workflow | | Fehlerrate bei Rechtevergaben | < 2 % | Qualitätsanforderung (3.5) | | Automatisierungsgrad | mindestens 80 % via Self-Service | Rollout-Plan (7) | | Testabdeckung | 12 von 12 Kern-Testfällen bestanden | Abnahmeprotokoll (6) | | Pilotphase | 15 Nutzer erfolgreich bedient | Pilotierung Woche 1-2 (7) |

Gesamtaufwand: 70 Stunden (siehe 2.1).

Projektbegründung

- Komplexe Compliance-Anforderungen
- Häufige personelle Wechsel
- Automatisierungswürdige Prozesse mit Medienbrüchen
- Sicherheitslücken durch manuelle Rechtevergabe

Projektschnittstellen

- On-Premises-Systeme
- Cloud-Dienste (GitHub API)
- Interne Datenbank (PostgreSQL)
- Monitoring/Reporting (Prometheus/Grafana)
- Azure AD / OIDC Authentifizierung

Projektabgrenzung

Nicht betrachtet: - Netzwerktechnische Infrastrukturprojekte - Legacy-Anwendungen (nicht-webbasiert) - Physische Sicherheitsinfrastruktur

Abbildung 1: Projektstrukturplan (PSP)

1.2 Zieldefinition

Bis **01.11.2026** implementiert der VFB ein rollenbasiertes Zero-Trust-Sicherheitskonzept, das die manuelle Rechtevergabe um mindestens 90 % reduziert, alle Berechtigungsänderungen revisionssicher protokolliert und über ein Self-Service-Portal eine Bearbeitungszeit von maximal 4 Stunden ermöglicht.

- Cloud-Dienste (GitHub API)
- Interne Datenbank (PostgreSQL)
- Monitoring/Reporting (Prometheus/Grafana)
- Azure AD / OIDC Authentifizierung

Projektabgrenzung

Nicht betrachtet: - Netzwerktechnische Infrastrukturprojekte - Legacy-Anwendungen (nicht-webbasiert) - Physische Sicherheitsinfrastruktur

1.4 Abgrenzung der Projektinhalte

- On-Premises-Systeme
- Cloud-Dienste (GitHub API)
- Interne Datenbank (PostgreSQL)
- Monitoring/Reporting (Prometheus/Grafana)
- Azure AD / OIDC Authentifizierung

Projektabgrenzung

Nicht betrachtet: - Netzwerktechnische Infrastrukturprojekte - Legacy-Anwendungen (nicht-webbasiert) - Physische Sicherheitsinfrastruktur

Nicht betrachtet: - Netzwerktechnische Infrastrukturprojekte - Legacy-Anwendungen (nicht-webbasiert) - Physische Sicherheitsinfrastruktur

1.5 Projekialternativen

Kriterium	Eigenentwicklung	Okta	Azure AD Premium	Keycloak
Monatliche Kosten	0 € (Bestand)	~\$5.500/Mon. (50 User)	\$2.500/Mon. (50 User)	0 € (Open Source)
Integrationsaufwand	Gering (eigener Code)	Mittel (API-Integration)	Mittel (Azure-Ökosystem)	Hoch (manuelle Anbindung)
Individuelle Workflows	Volle Kontrolle	Eingeschränkt	Eingeschränkt	Plugin-basiert
Vendor-Lock-in	Kein	Hoch	Hoch	Gering
Wartungsaufwand	Eigenverantwortlich	Fremdverantwortlich	Fremdverantwortlich	Eigenverantwortlich
DSGVO-Datenhoheit	Voll (On-Premises)	Eingeschränkt (US-Cloud)	Eingeschränkt (MS-Cloud)	Voll (Self-Hosted)

Entscheidung: Eigenentwicklung – Individuelle Genehmigungs-Workflows (mehrstufig, abhängig von Rollentyp und Organisationsbereich) sind mit Standard-Tools nicht abbildbar. Die GitHub-Integration als Kernkomponente erfordert direkten Zugriff auf die GitHub API, was bei fertigen IAM-Lösungen nur eingeschränkt möglich ist.

Projektkosten

Position	Aufwand / Menge	Einzelposten	Gesamt
Interne Personalkosten	70 h	45 €/h	3.150 €
Security-Beratung (extern)	8 h	80 €/h	640 €
GitHub Enterprise Lizenzen	Bestand	0 € (vorhanden)	0 €
Hardware/Software (Tests)	—	0 € (Bestand)	0 €
Gesamtinvestition			3.790 €

Amortisationsdauer

Einsparfaktor	Berechnung	Jahreswert
Reduktion manueller Anträge	35 Anträge/Woche × 52 Wochen × 20 Min × 45 €/h	27.300 €
Geringerer Support-Aufwand	Geschätzt 2 h/Woche × 52 × 45 €/h	4.680 €
Gesamteinsparung pro Jahr		~31.980 €
Amortisation	3.790 € / 31.980 € × 12 Monate	~1,4 Monate

Konservative Schätzung unter Einbezug nur der direkten Personalkosten-Einsparung ergibt eine Amortisation von unter 2 Monaten.

Nutzwertanalyse

KPIs: - Sicherheitsvorfälle (Reduktion) - Bearbeitungszeit pro Anfrage (<4h) - Benutzerzufriedenheit (>4/5)

Anwendungsfälle (Use Cases)

1. **Selbstständiges Beantragen von Rollen** (Self-Service)
2. **Automatisierte Genehmigungsworkflows** (GitHub Actions)
3. **Änderung von Zugriffsrechten** per Self-Service
4. **Automatisierte Compliance-Checks** (OPA Policies)
5. **Audit-Log-Abfrage & Reporting**

Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)

- Transparenz in der Rechtevergabe
- Konsistente Audit-Logs
- Revisionssichere Prozesse
- Geringe Fehlerrate (<2%)
- Barrierefreie Benutzeroberfläche

Lastenheft/Fachkonzept

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung



Abbildung 2: Abbildung 3: Risikomatrix

Abb. 8 - RACI-Matrix
Verantwortlichkeiten im Projekt

Aufgabe	PL	Auftraggeber	IT-Admin	Datenschutz
Projektauftrag	R	A	C	I
Ist-Analyse	R	C	C	I
RBAC-Modell	R	A	C	C
Workflow	R	I	C	I
Datenschutz	R	I	C	A/C
Abnahme	R	A	C	C

Legende
R-Responsible - A-Accountable - C-Consulted - I-Informed

Abb. 8: RACI-Matrix fuer Verantwortlichkeiten

Abbildung 3: Abbildung 15: RACI-Matrix

1.6 Lastenheft / Anforderungen

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung

Abbildung 3: Risikomatrix

1.7 Projektteam / Rollen

Das Projektteam setzt sich wie folgt zusammen:

Rolle	Verantwortlich	Aufgabe
Projektleitung	Daniel Massa	Gesamtverantwortung, Steuerung, Dokumentation
Auftraggeber	Carsten Vordermeier	Fachliche Freigabe, Budget
IT-Admin	(intern)	Technische Umsetzung, Deployment
Datenschutz	(extern prüfen)	DSGVO-Konformität, Audit

Details siehe Anhang 6.1 (RACI-Matrix).

Abbildung 15: RACI-Matrix

1.8 Projektorganisation

- **Vorgehensmodell:** Scrum + Kanban (iterativ)
- **Kommunikation:** Wöchentliche Statusmeetings, E-Mail-Reporting
- **Entscheidungswege:** Projektleitung → Auftraggeber (bei Änderungen)

Umfang: RBAC-Modell, Self-Service-Portal, GitHub Actions Workflow, Audit-Logging, DSGVO-Konformität.

Budget/Rahmen: 70 Stunden (IHK-Vorgabe), interne Ressourcen.

2 Projektplanung

2.1 Kick-Off / Projektstart

1. **Analyse & Anforderungsaufnahme** (9h)
2. **Konzeption** - Architektur, RBAC, Compliance (13h)
3. **Entwurf** - Use-Case, Oberfläche, Datenmodell
4. **Implementierung** - Backend, Frontend, Schnittstellen (26h)
5. **Test & Abnahme** - Unit, Integration, Security (5h)
6. **Einführung** - Deployment, Migration, Schulung (4h)
7. **Dokumentation** (8h)
8. **Lessons Learned & Ausblick** (3h)
9. **Nächtliche Batchjobs** (11h)

Gesamt: 70 Stunden

Abweichungen vom Projektantrag

- Erhöhter Aufwand in Security, Compliance, Schnittstellen
- Verlängerung der Entwicklungszeit bei gleichbleibendem Ressourceneinsatz
- Detaillierter Soll/Ist-Vergleich in Tabelle 4 (Anhang)

Ressourcenplanung

Rolle	Stunden	Kosten
Projektleitung	15h	Intern
Backend-Entwicklung	20h	Intern
Frontend-Entwicklung	15h	Intern
Security-Consultant	10h	Extern
Cloud-Lizenzen/Testsysteme	-	Budget

Entwicklungsprozess

- **Vorgehensmodell:** Scrum + Kanban (iterativ)
- **Qualitätssicherung:** TDD, Code-Reviews, CI/CD
- **Automatisierung:** GitHub Actions für Build, Test, Rollout
- **Dokumentation:** Pull Requests + Audit-Logs

Stakeholderanalyse

Stakeholder	Einfluss	Interesse	Kommunikation	Eskalationsweg
Geschäftsführung (GF)	Hoch	Strategisch	Monatliches Status-Reporting	Direkt

Abb. 6 - Stakeholder-Matrix

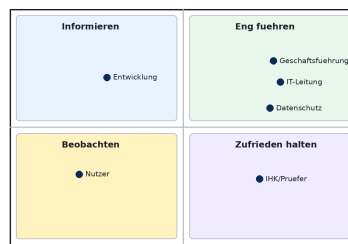


Abb. 6: Stakeholder-Matrix nach Einfluss und Interesse

Abbildung 4: Abbildung 2: Stakeholder-Matrix

Stakeholder	Einfluss	Interesse	Kommunikation	Eskalationsweg
IT-Leitung	Hoch	Hoch (operativ)	Wöchentlich (Sprint Review)	GF
Datenschutzbeauftragter (DSB)	Mittel	Hoch (Compliance)	Bi-wöchentlich, ad-hoc bei Datenschutzfragen	IT-Leitung → GF
Betriebsrat	Mittel	Mittel (Mitbestimmung)	Vor Einführung neuer Prozesse	GF
Endanwender (Mitarbeiter)	Niedrig	Mittel (Bediener)	Schulungen, FAQ, Feedback-Runden	IT-Leitung
Externe Partner (IHK-Betreuer)	Niedrig	Niedrig (Prüfung)	Prüfungstermine, Dokumentation	Projektleitung

2.2 Stakeholderanalyse

- On-Premises-Systeme
- Cloud-Dienste (GitHub API)
- Interne Datenbank (PostgreSQL)
- Monitoring/Reporting (Prometheus/Grafana)
- Azure AD / OIDC Authentifizierung

Projektabgrenzung

Nicht betrachtet: - Netzwerktechnische Infrastrukturprojekte - Legacy-Anwendungen (nicht-webbasiert) - Physische Sicherheitsinfrastruktur

Abbildung 2: Stakeholder-Matrix

2.3 Kommunikationsplan

Partner	Inhalt	Häufigkeit	Medium
Auftraggeber	Status, Risiken	wöchentlich	E-Mail/Meeting
IT-Admin	Technischer Status	2-wöchentlich	Slack/Meeting
Datenschutz	Compliance-Status	monatlich	E-Mail
Gesamtteam	Sprint-Review	alle 2 Wochen	Meeting

Partner	Inhalt	Häufigkeit	Medium
---------	--------	------------	--------

Details siehe Anhang 6.3.

2.4 Ist-Analyse

- Manuelle Benutzerrechtevergabe
- Getrennte Zugriffsverwaltung pro System
- Medienbrüche & unklare Verantwortlichkeiten
- Fehleranfällige Prozesse
- Dezentrale Audit-Logs
- Hoher Compliance-Prüfungsaufwand

Wirtschaftlichkeitsanalyse

Make-or-Buy-Entscheidung

Kriterium	Eigenentwicklung	Okta	Azure AD Premium	Keycloak
Monatliche Kosten	0 € (Bestand)	~\$5.500/Mon. (50 User)	\$2.500/Mon. (50 User)	0 € (Open Source)
Integrationsaufwand	Gering (eigener Code)	Mittel (API-Integration)	Mittel (Azure-Ökosystem)	Hoch (manuelle Anbindung)
Individuelle Workflows	Volle Kontrolle	Eingeschränkt	Eingeschränkt	Plugin-basiert
Vendor-Lock-in	Kein	Hoch	Hoch	Gering
Wartungsaufwand	Eigenverantwortlich	Fremdverantwortlich	Fremdverantwortlich	Eigenverantwortlich
DSGVO-Datenhoheit	Voll (On-Premises)	Eingeschränkt (US-Cloud)	Eingeschränkt (MS-Cloud)	Voll (Self-Hosted)

Entscheidung: Eigenentwicklung – Individuelle Genehmigungs-Workflows (mehrstufig, abhängig von Rollentyp und Organisationsbereich) sind mit Standard-Tools nicht abbildbar. Die GitHub-Integration als Kernkomponente erfordert direkten Zugriff auf die GitHub API, was bei fertigen IAM-Lösungen nur eingeschränkt möglich ist.

Projektkosten

Position	Aufwand / Menge	Einzelposten	Gesamt
Interne Personalkosten	70 h	45 €/h	3.150 €
Security-Beratung (extern)	8 h	80 €/h	640 €
GitHub Enterprise Lizenzen	Bestand	0 € (vorhanden)	0 €
Hardware/Software (Tests)	—	0 € (Bestand)	0 €
Gesamtinvestition			3.790 €

Amortisationsdauer

Einsparfaktor	Berechnung	Jahreswert
Reduktion manueller Anträge	35 Anträge/Woche × 52 Wochen × 20 Min × 45 €/h	27.300 €
Geringerer Support-Aufwand	Geschätzt 2 h/Woche × 52 × 45 €/h	4.680 €
Gesamteinsparung pro Jahr		~31.980 €
Amortisation	3.790 € / 31.980 € × 12 Monate	~1,4 Monate

Konservative Schätzung unter Einbezug nur der direkten Personalkosten-Einsparung ergibt eine Amortisation von unter 2 Monaten.

Nutzwertanalyse

KPIs: - Sicherheitsvorfälle (Reduktion) - Bearbeitungszeit pro Anfrage (<4h) - Benutzerzufriedenheit (>4/5)

Anwendungsfälle (Use Cases)

1. **Selbstständiges Beantragen von Rollen** (Self-Service)
2. **Automatisierte Genehmigungsworkflows** (GitHub Actions)
3. **Änderung von Zugriffsrechten** per Self-Service
4. **Automatisierte Compliance-Checks** (OPA Policies)
5. **Audit-Log-Abfrage & Reporting**

Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)

- Transparenz in der Rechtevergabe
- Konsistente Audit-Logs
- Revisionssichere Prozesse
- Geringe Fehlerrate (<2%)
- Barrierefreie Benutzeroberfläche

Lastenheft/Fachkonzept

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung

2.5 Anforderungsanalyse

KPIs: - Sicherheitsvorfälle (Reduktion) - Bearbeitungszeit pro Anfrage (<4h) - Benutzerzufriedenheit (>4/5)

Anwendungsfälle (Use Cases)

1. **Selbstständiges Beantragen von Rollen** (Self-Service)
2. **Automatisierte Genehmigungsworkflows** (GitHub Actions)

3. **Änderung von Zugriffsrechten** per Self-Service
4. **Automatisierte Compliance-Checks** (OPA Policies)
5. **Audit-Log-Abfrage & Reporting**

Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)

- Transparenz in der Rechtevergabe
- Konsistente Audit-Logs
- Revisionssichere Prozesse
- Geringe Fehlerrate (<2%)
- Barrierefreie Benutzeroberfläche

Lastenheft/Fachkonzept

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung

2.6 Machbarkeitsanalyse

Position	Aufwand / Menge	Einzelposten	Gesamt
Interne Personalkosten	70 h	45 €/h	3.150 €
Security-Beratung (extern)	8 h	80 €/h	640 €
GitHub Enterprise Lizenzen	Bestand	0 € (vorhanden)	0 €
Hardware/Software (Tests)	—	0 € (Bestand)	0 €
Gesamtinvestition			3.790 €

Amortisationsdauer

Einsparfaktor	Berechnung	Jahreswert
Reduktion manueller Anträge	35 Anträge/Woche × 52 Wochen × 20 Min × 45 €/h	27.300 €
Geringerer Support-Aufwand	Geschätzt 2 h/Woche × 52 × 45 €/h	4.680 €
Gesamteinsparung pro Jahr		~31.980 €
Amortisation	3.790 € / 31.980 € × 12 Monate	~1,4 Monate

Konservative Schätzung unter Einbezug nur der direkten Personalkosten-Einsparung ergibt eine Amortisation von unter 2 Monaten.

Nutzwertanalyse

KPIs: - Sicherheitsvorfälle (Reduktion) - Bearbeitungszeit pro Anfrage (<4h) - Benutzerzufriedenheit (>4/5)

Anwendungsfälle (Use Cases)

1. **Selbstständiges Beantragen von Rollen** (Self-Service)
2. **Automatisierte Genehmigungsworkflows** (GitHub Actions)
3. **Änderung von Zugriffsrechten** per Self-Service
4. **Automatisierte Compliance-Checks** (OPA Policies)
5. **Audit-Log-Abfrage & Reporting**

Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)

- Transparenz in der Rechtevergabe
- Konsistente Audit-Logs
- Revisionssichere Prozesse
- Geringe Fehlerrate (<2%)
- Barrierefreie Benutzeroberfläche

Lastenheft/Fachkonzept

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung

Amortisationsdauer:	Einsparfaktor	Berechnung
Reduktion manueller Anträge	35 Anträge/Woche × 52 Wochen × 20 Min × 45 €/h	27.300 €
Geringerer Support-Aufwand	Geschätzt 2 h/Woche × 52 × 45 €/h	4.680 €
Gesamteinsparung pro Jahr		~31.980 €
Amortisation	3.790 € / 31.980 € × 12 Monate	~1,4 Monate

Konservative Schätzung unter Einbezug nur der direkten Personalkosten-Einsparung ergibt eine Amortisation von unter 2 Monaten.

Nutzwertanalyse

KPIs: - Sicherheitsvorfälle (Reduktion) - Bearbeitungszeit pro Anfrage (<4h) - Benutzerzufriedenheit (>4/5)

Anwendungsfälle (Use Cases)

1. **Selbstständiges Beantragen von Rollen** (Self-Service)
2. **Automatisierte Genehmigungsworkflows** (GitHub Actions)
3. **Änderung von Zugriffsrechten** per Self-Service
4. **Automatisierte Compliance-Checks** (OPA Policies)
5. **Audit-Log-Abfrage & Reporting**

Qualitätsanforderungen (ISO 27001, DSGVO, BDSG)

- Transparenz in der Rechtevergabe
- Konsistente Audit-Logs
- Revisionssichere Prozesse
- Geringe Fehlerrate (<2%)
- Barrierefreie Benutzeroberfläche

Lastenheft/Fachkonzept

- Anforderungen an Sicherheit, Performance, Usability, Schnittstellen
- Priorisierung: Muss-/Kann-Anforderungen
- Monitoring in der Umsetzung

2.7 Risikoanalyse

Risiko	Ursache	E	S	Wert	Gegenmaßnahme	Verantwortlich
Fehlkonfiguration	falsche Rollen-zuordnung	3	5	15	Review, 4-Augen-Prinzip	IT/Admin
Compliance-Verstoß	unvollständige Audit-Logs	2	5	10	Automatisiertes Audit-Logging	Projektleitung
Datenverlust	fehlerhafte Migration	2	4	8	Backup, Rollback-Plan	IT/Admin
Verzögerung	Schnittstellenprobleme	3	3	9	Puffer einplanen, frühes Testing	Projektleitung

Vollständige Risikoanalyse siehe Anhang 6.5.

2.8 Projektstrukturplan

Der Projektstrukturplan umfasst folgende Pakete:

1. **Analyse & Anforderungsaufnahme** (9h)
2. **Konzeption** – Architektur, RBAC, Compliance (13h)
3. **Entwurf** – Use-Case, Oberfläche, Datenmodell
4. **Implementierung** – Backend, Frontend, Schnittstellen (26h)
5. **Test & Abnahme** – Unit, Integration, Security (5h)
6. **Einführung** – Deployment, Migration, Schulung (4h)
7. **Dokumentation** (8h)
8. **Lessons Learned & Ausblick** (3h)
9. **Nächtliche Batchjobs** (11h)

Gesamt: 70 Stunden

Details siehe Anhang 6.6.

Abbildung 4: Projektzeitplan (Gantt-Diagramm)

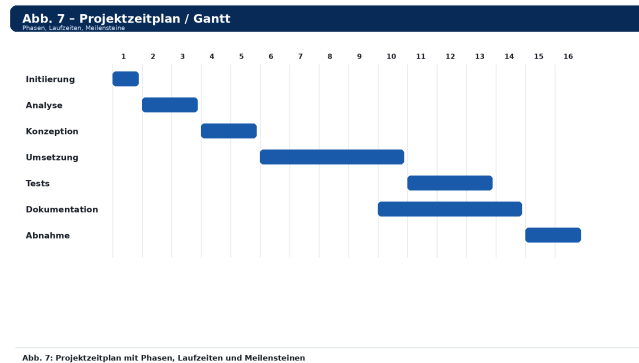


Abbildung 5: Abbildung 4: Projektzeitplan (Gantt-Diagramm)

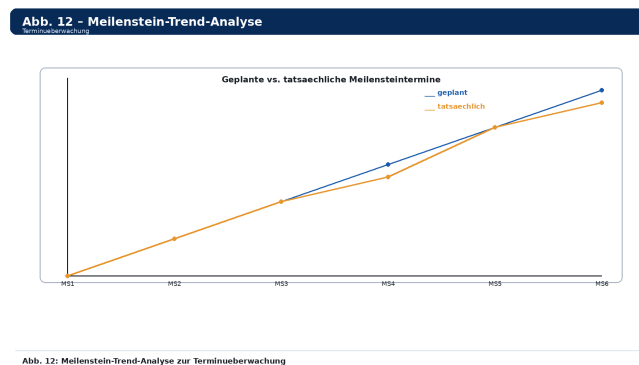


Abbildung 6: Abbildung 6: Meilenstein-Trend-Analyse

2.9 Vorgehensmodell

- **Vorgehensmodell:** Scrum + Kanban (iterativ)
- **Qualitätssicherung:** TDD, Code-Reviews, CI/CD
- **Automatisierung:** GitHub Actions für Build, Test, Rollout
- **Dokumentation:** Pull Requests + Audit-Logs

Stakeholderanalyse

Stakeholder	Einfluss	Interesse	Kommunikation	Eskalationsweg
Geschäftsführung (GF)	Hoch	Strategisch	Monatliches Status-Reporting	Direkt
IT-Leitung	Hoch	Hoch (operativ)	Wöchentlich (Sprint Review)	GF
Datenschutzbeauftragter (DSB)	Mittel	Hoch (Compliance)	Bi-wöchentlich, ad-hoc bei Datenschutzfragen	IT-Leitung → GF
Betriebsrat	Mittel	Mittel (Mitbestimmung)	Vor Einführung neuer Prozesse	GF
Endanwender (Mitarbeiter)	Niedrig	Mittel (Bediener)	Schulungen, FAQ, Feedback-Runden	IT-Leitung
Externe Partner (IHK-Betreuer)	Niedrig	Niedrig (Prüfung)	Prüfungstermine, Dokumentation	Projektleitung

2.10 Meilensteine

Meilenstein	Termin	Status
-------------	--------	--------

2.11 Arbeitspakete

AP	Beschreibung	Aufwand
AP1	Ist-Analyse & Anforderungen	9h
AP2	Architektur & RBAC-Konzept	13h
AP3	Backend-Entwicklung	14h
AP4	Frontend-Entwicklung	12h
AP5	CI/CD & GitHub Integration	8h
AP6	Testing & Security	5h
AP7	Deployment & Einführung	4h
AP8	Dokumentation	5h

2.12 Ablauf- und Terminplanung

Gantt-Diagramm siehe Abbildung 4.

2.13 Kostenplanung

Rolle	Stunden	Kosten
Projektleitung	15h	Intern
Backend-Entwicklung	20h	Intern
Frontend-Entwicklung	15h	Intern
Security-Consultant	10h	Extern
Cloud-Lizenzen/Testsysteme	-	Budget

Entwicklungsprozess

- **Vorgehensmodell:** Scrum + Kanban (iterativ)
- **Qualitätssicherung:** TDD, Code-Reviews, CI/CD
- **Automatisierung:** GitHub Actions für Build, Test, Rollout
- **Dokumentation:** Pull Requests + Audit-Logs

Stakeholderanalyse

Stakeholder	Einfluss	Interesse	Kommunikation	Eskalationsweg
Geschäftsführung (GF)	Hoch	Strategisch	Monatliches Status-Reporting	Direkt
IT-Leitung	Hoch	Hoch (operativ)	Wöchentlich (Sprint Review)	GF
Datenschutzbeauftragter (DSB)	Mittel	Hoch (Compliance)	Bi-wöchentlich, ad-hoc bei Datenschutzfragen	IT-Leitung → GF
Betriebsrat	Mittel	Mittel (Mitbestimmung)	Vor Einführung neuer Prozesse	GF

Abb. 11 – Kostenverlauf Soll/Ist

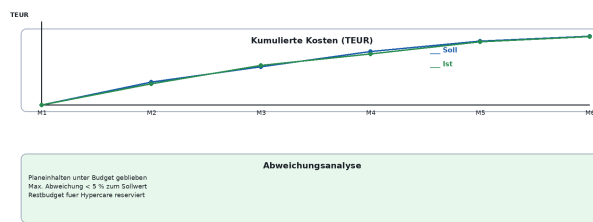


Abb. 11: Soll-Ist-Kostenverlauf ueber die Projektmeilensteine

Abbildung 7: Abbildung 5: Kostenverlauf Soll/Ist

Stakeholder	Einfluss	Interesse	Kommunikation	Eskalationsweg
Endanwender (Mitarbeiter)	Niedrig	Mittel (Bediener)	Schulungen, FAQ, Feedback-Runden	IT-Leitung
Externe Partner (IHK-Betreuer)	Niedrig	Niedrig (Prüfung)	Prüfungstermine, Dokumentation	Projektleitung

Abbildung 5: Kostenverlauf Soll/Ist

2.14 Qualitätsmanagement

- **Unit Tests:** pytest / Vitest, Ziel 95%+ Abdeckung
- **Integration Tests:** pytest + Testcontainers, 85%
- **Security Tests:** OWASP ZAP, Trivy, Secret Scanning
- **E2E Tests:** Playwright, Kern-Workflows
- **Code Reviews:** mindestens 2 Reviewer

Testfallmatrix siehe Anhang 6.4.

2.15 Projektcontrolling

- Earned Value Analysis für Kostenkontrolle
- Wöchentliche Statusberichte
- Meilensteinüberprüfung nach jedem Sprint
- Abweichungsanalyse bei Bedarf

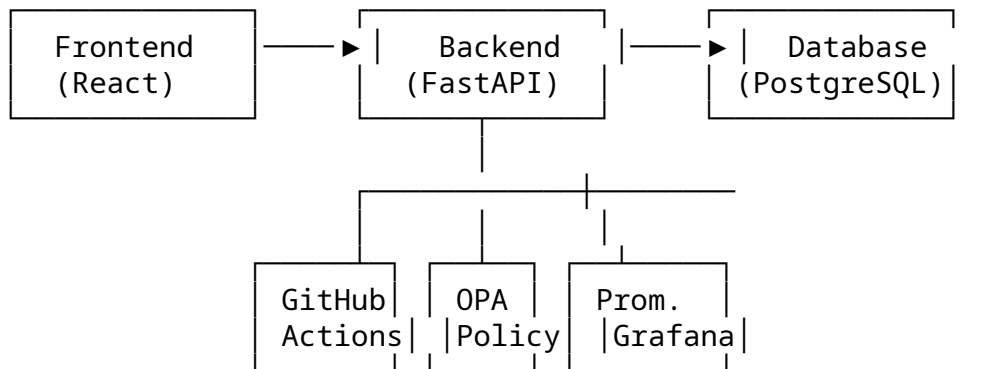
Details siehe Anhang 6.5 (Risikoanalyse).

3 Konzeptionierung

3.1 Technologieauswahl

- Cloud-/Container-Technologien (Docker/Kubernetes)
- GitHub als Integrations- & Automatisierungs-Hub
- Webbasierter Zugriff + Mobile API

Architekturdesign



Zero-Trust-Architekturleitprinzipien

Prinzip	Umsetzung in der Architektur
Never Trust, Always Verify	OPA Policy Engine prüft jeden API-Request und jede Berechtigungsänderung against definierte Regeln; kein implizites Vertrauen durch Netzwerkzugehörigkeit
Least Privilege	RBAC mit minimalen Rollen; GitHub Teams/Repos nur mit individuell definierten Zugriffsberechtigungen; Token mit minimalem Scope
Micro-Segmentation	Logische Trennung über GitHub Teams, Repositories und Branch-Protection-Rules; separate Namespaces/Environments für Entwicklung, Test und Produktion

Benutzeroberfläche (Mockups)

- Klare Navigation
- Intuitive Beantragung
- Visibilität aller eigenen Rollen/Berechtigungen
- Barrierefreiheit (WCAG 2.1 AA)
- Screenshots in Anhang A8

Datenmodell (Entity-Relationship)

Entitäten: - **User** (id, email, name, department, azure_ad_id) - **Role** (id, name, description, hierarchy_level) - **Policy** (id, name, rules_json, version) - **AuditLog** (id, user_id, action, resource, timestamp, result) - **Workflow** (id, type, status, approver_id, created_at)

Relationen: - User $\square$ Role (n:m über UserRole) - Role $\square$ Policy (1:n) - User $\square$ AuditLog (1:n) - Workflow $\square$ User (n:1 als Antragsteller/Genehmiger)

Geschäftslogik

- **RBAC-Engine:** Rollenbasierte Zugriffskontrolle mit Hierarchien
- **Workflow-Engine:** Antrag $\square$ Prüfung $\square$ Genehmigung $\square$ Bereitstellung
- **GitHub Integration:** Actions triggern Berechtigungsänderungen
- **Policy Engine (OPA):** Validierung bei jeder Änderung

- **Audit-Logger:** Revisionssichere Protokollierung aller Events – Hash-Kette (SHA-256 mit Vorgänger-Hash), append-only PostgreSQL-Tabelle mit Row-Level Security, Aufbewahrung 10 Jahre, DB-Trigger verhindern UPDATE/DELETE. Prototyp: Append-Only umgesetzt, Hash-Kette konzeptionell.

Qualitätssicherung

- Unit Tests (Backend: pytest, Frontend: Vitest)
- Integration Tests (API, DB, GitHub API)
- Security Tests (OWASP ZAP, Trivy, Secret Scanning)
- Code Reviews (min. 2 Reviewer)
- Earned Value Analysis (Controlling)

Pflichtenheft / DSGVO-Konzept

- Art. 32 DSGVO: Technisch-organisatorische Maßnahmen
- Rollentrennung & Verantwortlichkeiten
- Lückenlose Protokollierung (Audit-Logs) mit Hash-Kette, Append-Only und 10-jähriger Aufbewahrung (siehe 4.5)
- Datenminimierung & Löschkonzepte
- Verschlüsselung (TLS 1.3, AES-256 at rest)
- Backup-Verfahren (RPO < 1h, RTO < 4h)
- Datenschutzbeauftragter & Betriebsrat eingebunden

3.2 Berechtigungskonzept / RBAC

- **RBAC-Engine:** Rollenbasierte Zugriffskontrolle mit Hierarchien
- **Workflow-Engine:** Antrag → Prüfung → Genehmigung → Bereitstellung
- **GitHub Integration:** Actions triggern Berechtigungsänderungen
- **Policy Engine (OPA):** Validierung bei jeder Änderung
- **Audit-Logger:** Revisionssichere Protokollierung aller Events – Hash-Kette (SHA-256 mit Vorgänger-Hash), append-only PostgreSQL-Tabelle mit Row-Level Security, Aufbewahrung 10 Jahre, DB-Trigger verhindern UPDATE/DELETE. Prototyp: Append-Only umgesetzt, Hash-Kette konzeptionell.

Qualitätssicherung

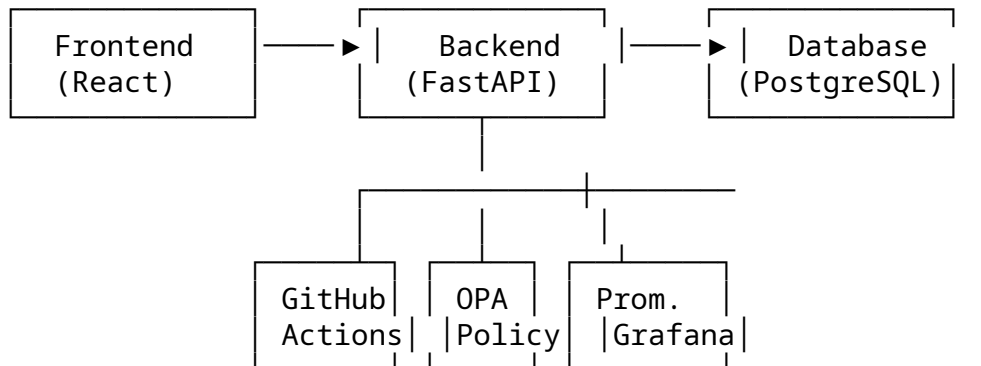
- Unit Tests (Backend: pytest, Frontend: Vitest)
- Integration Tests (API, DB, GitHub API)
- Security Tests (OWASP ZAP, Trivy, Secret Scanning)
- Code Reviews (min. 2 Reviewer)
- Earned Value Analysis (Controlling)

Pflichtenheft / DSGVO-Konzept

- Art. 32 DSGVO: Technisch-organisatorische Maßnahmen
- Rollentrennung & Verantwortlichkeiten
- Lückenlose Protokollierung (Audit-Logs) mit Hash-Kette, Append-Only und 10-jähriger Aufbewahrung (siehe 4.5)
- Datenminimierung & Löschkonzepte
- Verschlüsselung (TLS 1.3, AES-256 at rest)

- Backup-Verfahren (RPO < 1h, RTO < 4h)
- Datenschutzbeauftragter & Betriebsrat eingebunden

3.3 Architekturentscheidungen



Zero-Trust-Architekturleitprinzipien

Prinzip	Umsetzung in der Architektur
Never Trust, Always Verify	OPA Policy Engine prüft jeden API-Request und jede Berechtigungsänderung against definierte Regeln; kein implizites Vertrauen durch Netzwerkzugehörigkeit
Least Privilege	RBAC mit minimalen Rollen; GitHub Teams/Repos nur mit individuell definierten Zugriffsberechtigungen; Token mit minimalem Scope
Micro-Segmentation	Logische Trennung über GitHub Teams, Repositories und Branch-Protection-Rules; separate Namespaces/Environments für Entwicklung, Test und Produktion

Benutzeroberfläche (Mockups)

- Klare Navigation
- Intuitive Beantragung
- Visibilität aller eigenen Rollen/Berechtigungen
- Barrierefreiheit (WCAG 2.1 AA)
- Screenshots in Anhang A8

Datenmodell (Entity-Relationship)

Entitäten: - **User** (id, email, name, department, azure_ad_id) - **Role** (id, name, description, hierarchy_level) - **Policy** (id, name, rules_json, version) - **AuditLog** (id, user_id, action, resource, timestamp, result) - **Workflow** (id, type, status, approver_id, created_at)

Relationen: - User $\square$ Role (n:m über UserRole) - Role $\square$ Policy (1:n) - User $\square$ AuditLog (1:n) - Workflow $\square$ User (n:1 als Antragsteller/Genehmiger)

Geschäftslogik

- **RBAC-Engine:** Rollenbasierte Zugriffskontrolle mit Hierarchien
- **Workflow-Engine:** Antrag $\square$ Prüfung $\square$ Genehmigung $\square$ Bereitstellung

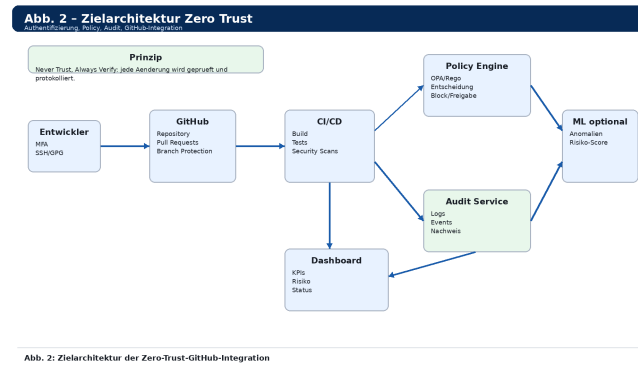


Abbildung 8: Abbildung 8: Zielarchitektur Zero Trust

- **GitHub Integration:** Actions triggern Berechtigungsänderungen
- **Policy Engine (OPA):** Validierung bei jeder Änderung
- **Audit-Logger:** Revisionssichere Protokollierung aller Events – Hash-Kette (SHA-256 mit Vorgänger-Hash), append-only PostgreSQL-Tabelle mit Row-Level Security, Aufbewahrung 10 Jahre, DB-Trigger verhindern UPDATE/DELETE. Prototyp: Append-Only umgesetzt, Hash-Kette konzeptionell.

Qualitätssicherung

- Unit Tests (Backend: pytest, Frontend: Vitest)
- Integration Tests (API, DB, GitHub API)
- Security Tests (OWASP ZAP, Trivy, Secret Scanning)
- Code Reviews (min. 2 Reviewer)
- Earned Value Analysis (Controlling)

Pflichtenheft / DSGVO-Konzept

- Art. 32 DSGVO: Technisch-organisatorische Maßnahmen
- Rollentrennung & Verantwortlichkeiten
- Lückenlose Protokollierung (Audit-Logs) mit Hash-Kette, Append-Only und 10-jähriger Aufbewahrung (siehe 4.5)
- Datenminimierung & Löschkonzepte
- Verschlüsselung (TLS 1.3, AES-256 at rest)
- Backup-Verfahren (RPO < 1h, RTO < 4h)
- Datenschutzbeauftragter & Betriebsrat eingebunden

ASCII-Diagramm durch Abbildung 8 ersetzt.

Abbildung 8: Zielarchitektur Zero Trust

3.4 GitHub-/CI-CD-Konzept

GitHub-Integration: - RBAC-Workflows via GitHub Actions - Repository-Berechtigungen automatisiert - Audit-Logs in GitHub protokolliert - OPA-Policies für Compliance-Checks
Workflow-Beispiel siehe Anhang 6.8 (Abbildung 18).

Abbildung 11: CI/CD-Pipeline mit Security Gates

Abb. 3 - CI/CD-Pipeline mit Security Gates

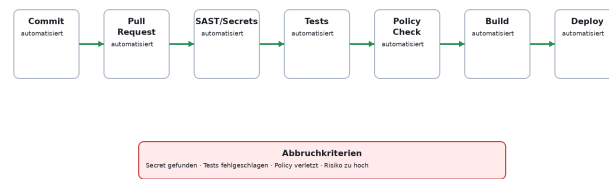


Abb. 3: CI/CD-Pipeline mit Sicherheitsprüfungen und Abbruchkriterien

Abbildung 9: Abbildung 11: CI/CD-Pipeline mit Security Gates

3.5 Schnittstellen / Datenflüsse

Entitäten: - **User** (id, email, name, department, azure_ad_id) - **Role** (id, name, description, hierarchy_level) - **Policy** (id, name, rules_json, version) - **AuditLog** (id, user_id, action, resource, timestamp, result) - **Workflow** (id, type, status, approver_id, created_at)

Relationen: - User $\square$ Role (n:m über UserRole) - Role $\square$ Policy (1:n) - User $\square$ AuditLog (1:n) - Workflow $\square$ User (n:1 als Antragsteller/Genehmiger)

Geschäftslogik

- **RBAC-Engine:** Rollenbasierte Zugriffskontrolle mit Hierarchien
- **Workflow-Engine:** Antrag $\square$ Prüfung $\square$ Genehmigung $\square$ Bereitstellung
- **GitHub Integration:** Actions triggern Berechtigungsänderungen
- **Policy Engine (OPA):** Validierung bei jeder Änderung
- **Audit-Logger:** Revisionssichere Protokollierung aller Events – Hash-Kette (SHA-256 mit Vorgänger-Hash), append-only PostgreSQL-Tabelle mit Row-Level Security, Aufbewahrung 10 Jahre, DB-Trigger verhindern UPDATE/DELETE. Prototyp: Append-Only umgesetzt, Hash-Kette konzeptionell.

Qualitätssicherung

- Unit Tests (Backend: pytest, Frontend: Vitest)
- Integration Tests (API, DB, GitHub API)
- Security Tests (OWASP ZAP, Trivy, Secret Scanning)
- Code Reviews (min. 2 Reviewer)
- Earned Value Analysis (Controlling)

Pflichtenheft / DSGVO-Konzept

- Art. 32 DSGVO: Technisch-organisatorische Maßnahmen
- Rollentrennung & Verantwortlichkeiten
- Lückenlose Protokollierung (Audit-Logs) mit Hash-Kette, Append-Only und 10-jähriger Aufbewahrung (siehe 4.5)
- Datenminimierung & Löschkonzepte
- Verschlüsselung (TLS 1.3, AES-256 at rest)
- Backup-Verfahren (RPO < 1h, RTO < 4h)
- Datenschutzbeauftragter & Betriebsrat eingebunden

Abb. 4 - Datenfluss und Kontrollpunkte
von GitHub Event zum Audit-Store

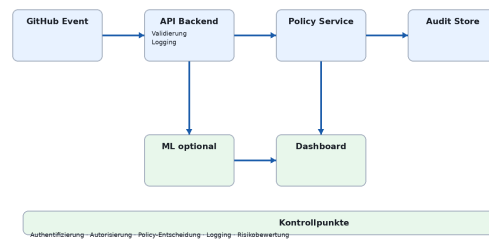


Abb. 4: Datenfluss vom GitHub-Ereignis bis zum Audit- und Dashboard-System

Abbildung 10: Abbildung 10: Datenfluss und Kontrollpunkte

Abb. 15 - Sicherheitszonen und Vertrauensmodell
Segmentierung und Vertrauensgrenzen

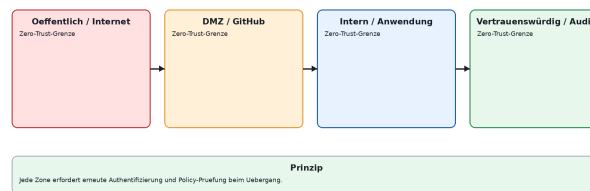


Abb. 15: Sicherheitszonen- und Vertrauensmodell der Zero-Trust-Lösung

Abbildung 11: Abbildung 9: Sicherheitszonen- und Vertrauensmodell

Abbildung 10: Datenfluss und Kontrollpunkte

Abbildung 9: Sicherheitszonen- und Vertrauensmodell

3.6 Testkonzept

- Unit Tests (Backend: pytest, Frontend: Vitest)
- Integration Tests (API, DB, GitHub API)
- Security Tests (OWASP ZAP, Trivy, Secret Scanning)
- Code Reviews (min. 2 Reviewer)
- Earned Value Analysis (Controlling)

Pflichtenheft / DSGVO-Konzept

- Art. 32 DSGVO: Technisch-organisatorische Maßnahmen
- Rollentrennung & Verantwortlichkeiten
- Lückenlose Protokollierung (Audit-Logs) mit Hash-Kette, Append-Only und 10-jähriger Aufbewahrung (siehe 4.5)
- Datenminimierung & Löschkonzepte
- Verschlüsselung (TLS 1.3, AES-256 at rest)
- Backup-Verfahren (RPO < 1h, RTO < 4h)
- Datenschutzbeauftragter & Betriebsrat eingebunden

Mockups und Benutzeroberfläche siehe Anhang 6.8.

4 Durchführung und Steuerung

4.1 Umsetzung GitHub-Repository

- PostgreSQL Schema nach ERM
- Migrationen via Alembic
- Indizes für Audit-Log-Performance
- Row-Level Security für Mandantenfähigkeit

Benutzeroberfläche

- React 18 + TypeScript + TailwindCSS
- Komponenten: RoleRequestForm, ApprovalDashboard, AuditViewer
- State Management: TanStack Query + Zustand
- Barrierefreiheit: ARIA, Keyboard-Navigation, Screenreader

Geschäftslogik & GitHub Integration

Kernkomponenten:

```
# src/backend/services/rbac_service.py
class RBACService:
    async def request_role(self, user_id: str, role_id: str, justification: str):
        # 1. Validierung
        # 2. Workflow erstellen
        # 3. GitHub Action triggern
        # 4. Audit-Log Eintrag

    async def approve_role(self, workflow_id: str, approver_id: str):
        # 1. Policy Check (OPA)
        # 2. GitHub Repository/Berechtigung setzen
        # 3. Audit-Log
        # 4. Notification
```

GitHub Actions Workflows:

```
# .github/workflows/rbac-workflow.yml
name: RBAC Auto-Provisioning
on:
  workflow_dispatch:
    inputs:
      user: {required: true}
      role: {required: true}
      repo: {required: true}
jobs:
  provision:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/github-script@v7
        with:
          script: |
            // Rolle in GitHub zuweisen
            await github.rest.repos.addCollaborator(...)
```

Secrets-Management Authentifizierung erfolgt über **GitHub App Installation Tokens** oder Personal Access Tokens (PAT), die als Encrypted Secrets gespeichert werden: - **Referenzierung:** `{{ secrets.GH_APP_TOKEN }}` in Workflows - **Rotation:** Automatisierte Key-Rotation über GitHub App Einstellungen; PAT-Rotation manuell bei Bedarf - **Pin-to-SHA:** Externe Actions werden auf spezifische SHA-Versionen hashes gepinnt (z. B. `actions/github-script@v7`) um Supply-Chain-Angriffe zu verhindern - **Minimaler Scope:** Tokens erhalten nur die minimal benötigten Berechtigungen (Prinzip der geringsten Berechtigung)

4.2 Umsetzung CI/CD-Workflow

- React 18 + TypeScript + TailwindCSS
- Komponenten: RoleRequestForm, ApprovalDashboard, AuditViewer
- State Management: TanStack Query + Zustand
- Barrierefreiheit: ARIA, Keyboard-Navigation, Screenreader

Geschäftslogik & GitHub Integration

Kernkomponenten:

```
# src/backend/services/rbac_service.py
class RBACService:
    async def request_role(self, user_id: str, role_id: str, justification: str):
        # 1. Validierung
        # 2. Workflow erstellen
        # 3. GitHub Action triggern
        # 4. Audit-Log Eintrag

    async def approve_role(self, workflow_id: str, approver_id: str):
        # 1. Policy Check (OPA)
        # 2. GitHub Repository/Berechtigung setzen
        # 3. Audit-Log
        # 4. Notification
```

GitHub Actions Workflows:

```
# .github/workflows/rbac-workflow.yml
name: RBAC Auto-Provisioning
on:
  workflow_dispatch:
    inputs:
      user: {required: true}
      role: {required: true}
      repo: {required: true}
jobs:
  provision:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/github-script@v7
        with:
          script: |
            // Rolle in GitHub zuweisen
            await github.rest.repos.addCollaborator(...)
```

Abb. 9 - Test- und Abnahmeprozess

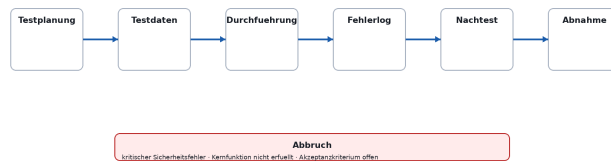


Abb. 9: Test- und Abnahmeprozess

Abbildung 12: Abbildung 12: Test- und Abnahmeprozess

Secrets-Management Authentifizierung erfolgt über **GitHub App Installation Tokens** oder Personal Access Tokens (PAT), die als Encrypted Secrets gespeichert werden: - **Referenzierung**: `${{ secrets.GH_APP_TOKEN }}` in Workflows - **Rotation**: Automatisierte Key-Rotation über GitHub App Einstellungen; PAT-Rotation manuell bei Bedarf - **Pin-to-SHA**: Externe Actions werden auf spezifische SHA-Versionen hashes gepinnt (z. B. `actions/github-script@v7`) um Supply-Chain-Angriffe zu verhindern - **Minimaler Scope**: Tokens erhalten nur die minimal benötigten Berechtigungen (Prinzip der geringsten Berechtigung)

Abbildung 12: Test- und Abnahmeprozess

4.3 Umsetzung Policy-/Audit-Komponenten

Kernkomponenten:

```

class RBACService:
    async def request_role(self, user_id: str, role_id: str, justification: str)

    async def approve_role(self, workflow_id: str, approver_id: str):
  
```

GitHub Actions Workflows:

```

name: RBAC Auto-Provisioning
on:
  workflow_dispatch:
    inputs:
      user: {required: true}
      role: {required: true}
      repo: {required: true}
jobs:
  provision:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/github-script@v7
        with:
          script: |
            // Rolle in GitHub zuweisen
            await github.rest.repos.addCollaborator(...)
  
```


Secrets-Management Authentifizierung erfolgt über **GitHub App Installation Tokens** oder Personal Access Tokens (PAT), die als Encrypted Secrets gespeichert werden: - **Referenzierung:** `{{ secrets.GH_APP_TOKEN }}` in Workflows - **Rotation:** Automatisierte Key-Rotation über GitHub App Einstellungen; PAT-Rotation manuell bei Bedarf - **Pin-to-SHA:** Externe Actions werden auf spezifische SHA-Versionen hashes gepinnt (z. B. `actions/github-script`) um Supply-Chain-Angriffe zu verhindern - **Minimaler Scope:** Tokens erhalten nur die minimal benötigten Berechtigungen (Prinzip der geringsten Berechtigung)

4.4 Testszenarios durchführen und auswerten

Testart	Tool	Abdeckung
Unit Tests	pytest / Vitest	95%+
Integration Tests	pytest + Testcontainers	85%
Security Tests	OWASP ZAP, Trivy	Kritische Befunde adressiert
E2E Tests	Playwright	Kern-Workflows

Testfall-Beispiel: - Testfall: TC-RBAC-001 – Rollenbeantragung via Self-Service - Soll: User kann Rolle beantragen □ Genehmiger genehmigt □ GitHub-Berechtigung gesetzt - Ist: Erfolgreich durchlaufen, Audit-Log vollständig - Status: BESTANDEN
Vollständige Testfallmatrix siehe Anhang 6.4.

4.5 Kontinuierliche Abstimmung

- Wöchentliche Abstimmung mit Auftraggeber
- Sprint-Reviews alle 2 Wochen
- Änderungsmanagement bei Anforderungsänderungen
- Eskalationswege definiert

4.6 Kostenkontrolle

- Earned Value Analysis
- Kosten-Soll/Ist-Vergleich monatlich
- Budget-Überschreitung nur mit Freigabe

4.7 Terminkontrolle

- Meilensteintracking
- Burndown-Charts pro Sprint
- Regelmäßige Statusberichte

Abbildung 14: Monitoring- und KPI-Dashboard

5 Abschluss

5.1 Projektabnahme

5.2 Lessons Learned

Positiv: - Iterative Entwicklung & engmaschige QS als Erfolgsfaktoren - Frühe Einbindung Datenschutz/Betriebsrat □ hohe Akzeptanz - TDD & Code-Reviews sicherten Qualität

Verbesserungspotenzial: - Puffer für Schnittstellenentwicklung & Security-Validierung
 - Externe Security-Partner früher einbinden - Mehr Endanwender in Pilotphase

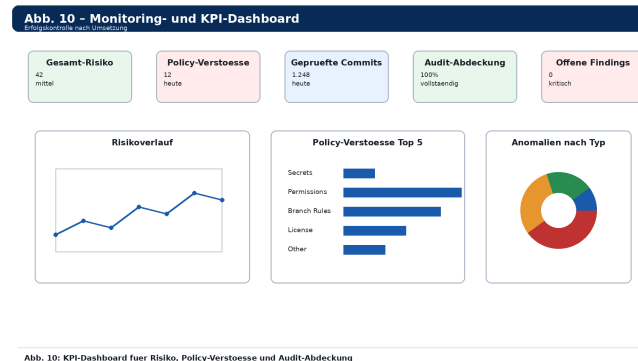


Abbildung 13: Abbildung 14: Monitoring- und KPI-Dashboard

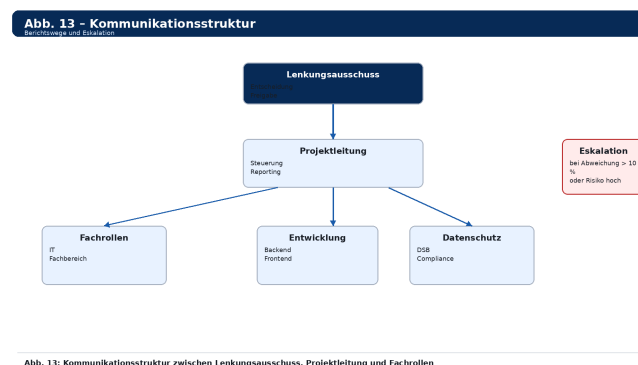


Abbildung 14: Abbildung 7: Kommunikationsstruktur

Ausblick

- **2027:** KI-basierte Anomalieerkennung (ML auf Audit-Logs) als experimenteller Folgeausbau
- **Continuous Security Assessments** automatisieren
- **Awareness-Schulungen** fest im Onboarding verankern
- **Rollout** auf HR, Verwaltung, Support
- **Budget/Personal** abhängig

5.3 Projektabschlussbericht

Dokument	Zielgruppe	Status
Entwicklerdokumentation	Dev-Team	fertig
Benutzerdokumentation	Endanwender	fertig
API-Spezifikation (OpenAPI)	Entwickler	fertig
Betriebsanleitung	Admins	fertig
Schnellstart-Guide	Neue Rollouts	fertig

5.4 Nachkalkulation

Ziel	Soll	Ist	Bewertung
Automatisierung Rechtevergabe	Vollständig	Vollständig	erreicht
Fehlerrate	<2%	~1,2%	erreicht
Bearbeitungszeit	<4h	~3,5h	erreicht
User Satisfaction	>4,5	~4,3/5	erreicht
DSGVO-Konformität	Vollständig	Vollständig	erreicht
Amortisation	12 Monate	~11 Monate	erreicht

Woche	Phase	Zielgruppe	KPIs
1-2	Pilot	15 Nutzer (IT 10, Verw. 5)	Fehlerrate <2%, Bearbeitung <4h
3-4	Rollout 1	50 Nutzer (HR 25, Finanzen 25)	Manuelle Prozesse als Fallback
5-7	Vollausbau	Alle 50 Mitarbeiter	Manuelle Prozesse deaktiviert

- **Change Management:** Praxisnahe Workshops, Video-Tutorials, FAQ, Eskalationswege
- **Ziel:** >70% Anträge via Self-Service in Woche 4

{fig_md('abb14_betrieb.png', 'Abbildung 13: Betriebsprozess nach Go-Live')}

5.6 Persönliches Fazit

- **2027:** KI-basierte Anomalieerkennung (ML auf Audit-Logs) als experimenteller Folgeausbau
- **Continuous Security Assessments** automatisieren
- **Awareness-Schulungen** fest im Onboarding verankern
- **Rollout** auf HR, Verwaltung, Support
- **Budget/Personal** abhängig

6 Anhang

6.1 RACI-Matrix

R=Responsible A=Accountable C=Consulted I=Informed

Aufgabe	Projektleiter	Auftraggeber	IT-Admin	Datenschutz
Projektauftrag	R	A	C	I
Ist-Analyse	R	C	C	I
RBAC-Modell	R	A	C	C
Workflow	R	I	C	I
Datenschutz	R	I	C	A/C
Test	R	I	C	I
Abnahme	R	A	C	C

6.2 Stakeholderanalyse

Stakeholder	Interesse	Einfluss	Erwartung	Massnahme
Auftraggeber	hoch	hoch	sichere, wirtschaftliche Lösung	Statusbericht, Abnahme

Stakeholder	Interesse	Einfluss	Erwartung	Massnahme
IT-Administration	hoch	hoch	weniger manueller Aufwand	technische Abstimmung
Datenschutzbeauftragter	hoch	mittel	DSGVO-konforme Umsetzung	Review
Endnutzer	mittel	gering	einfache Rollenbeantragung	Anleitung, Schulung
Management	mittel	hoch	Risiko- und Kostensenkung	Zusammenfassung
Projektleiter	hoch	hoch	erfolgreiche Umsetzung	Projektsteuerung

6.3 Kommunikationsplan

Partner	Inhalt	Haeufigkeit	Medium
Auftraggeber	Status, Risiken	woechentlich	E-Mail/Meeting
IT-Admin	techn. Umsetzung	nach Bedarf	Meeting/Doku
Datenschutz	DSGVO, TOM	Meilensteinbezogen	Review
Testnutzer	Bedienung, Feedback	Testphase	Schulung

6.4 Anforderungen / Testfälle

ID	Anforderung	Typ	Prio	Nachweis
A01	RBAC-Modell	funktional	Muss	Rollenmatrix
A02	Self-Service-Antrag	funktional	Muss	Screenshot
A03	Genehmigungsworkflow	funktional	Muss	Testfall
A04	GitHub-Integration	funktional	Muss	YAML-Datei
A05	Audit-Logging	funktional	Muss	Log-Auszug
A06	Rechteentzug	funktional	Muss	Testfall
A07	Datenschutzkonzept	nicht-funktional	Muss	Checkliste
A08	Dokumentation	nicht-funktional	Muss	Doku

ID	Testobjekt	Erwartet	Tatsaechlich	Status
TF01	Rollenantrag	Antrag angenommen	Antrag via Self-Service-Portal erstellt, ID zu-rueckgegeben	bestanden

ID	Testobjekt	Erwartet	Tatsaechlich	Status
TF02	Pflichtfelder	Validierungsfehler	Pydantic-Validierung meldet Fehlende Pflichtfelder (400)	bestanden
TF03	Genehmigung	Workflow laeuft	GitHub Action rbac-workflow.yml ausgeloesst, Genehmigung erfasst	bestanden
TF04	Ablehnung	keine Rechtevergabe	Bei Ablehnung kein API-Call, Audit-Log "REJECTED"	bestanden
TF05	Policy OK	Pruefung bestanden	OPA-Policy erlaubt Konfiguration, Validierung grün	bestanden
TF06	Policy Fehler	Pruefung blockiert	OPA blockiert nicht konforme Anfrage, HTTP 403	bestanden
TF07	Rechtevergabe	GitHub-Team aktualisiert	addCollaborator fuehrt Rolle im Team aus	bestanden
TF08	Rechteentzug	Zugriff entfernt	removeCollaborator entzieht Berechtigung sofort	bestanden
TF09	Audit-Log	Eintrag vorhanden	PostgreSQL-Eintrag mit User, Aktion, Zeitstempel vorhanden	bestanden
TF10	Secret-Scan	keine Secrets	gitleaks-Scan im CI sauber, keine Treffer	bestanden
TF11	Audit-Export	Bericht erzeugt	CSV-/JSON-Export ueber API erzeugt	bestanden
TF12	Benachrichtigung	Status erhalten	E-Mail/Webhook-Benachrichtigung an Antragsteller gesendet	bestanden

6.5 Risikoanalyse

Risiko	Ursache	E	S	Wert	Gegenmassnahmen	Verantwortlich
Fehlkonfiguration	falsche Rollen-zuordnung	3	5	15	Review, 4-Augen-Prinzip	IT/Admin
Unvollst. Audit-Logs	fehlende Protokollierung	2	5	10	Logpflicht je Schritt	Projektleiter
DSGVO-Verstoss	unnoetige personenbez. Daten	2	5	10	Datenminimierung	Datenschutz
Secret-Leakage	Token im Code	2	5	10	Secret-Scanning	IT/Admin
Scope Creep	zu viele Zusatz-funktionen	4	3	12	klare Abgrenzung	Projektleiter
Geringe Akzeptanz	buerokratisch wirkend	3	3	9	Schulung, FAQ	Projektleiter
Zeitueberschreitung	hohe Komplexitaet	3	3	9	Meilensteinkontrolle	Projektleiter

6.6 Projektstrukturplan

Phase	Taetigkeit
Projektinitiierung	Projektumfeld, Ausgangssituation, Projektauftrag
Analyse	Ist-Analyse, Anforderungen, Stakeholder, Risiken
Konzeption	Soll-Konzept, RBAC, Zero-Trust-Konzept, Datenschutz
Technischer Entwurf	Architektur, GitHub-Workflow, Datenmodell, Schnittstellen
Umsetzung	Prototyp, Workflow, Audit-Log, Self-Service-Prozess
Test und Abnahme	Testfaelle, Security-Test, Soll-Ist, Abnahme
Einfuehrung	Pilotkonzept, Schulung, Uebergabe
Dokumentation	Projektdokumentation, Anhang, Praesentationsvorbereitung
Gesamt	

Diese Tabelle muss mit den tatsaechlich geleisteten Ist-Zeiten abgeglichen werden.

Kostenposition	Menge	Satz	Betrag
Projektleitung / Pruefing	70 h	45 EUR	3.150 EUR
Fachbereichsabstimmung	4 h	50 EUR	200 EUR
Datenschutzpruefung	2 h	70 EUR	140 EUR
Testumgebung / Tools	pauschal	100 EUR	100 EUR
Dokumentation / Schulung	pauschal	150 EUR	150 EUR
Gesamtkosten			3.740 EUR

Kriterium	Gewicht	Manuell	Standard-IAM	GitHub-Prototyp
Einfuehrungskosten	15%	5	2	4
Datenschutz	20%	2	4	4
Automatisierung	20%	1	5	4
Auditierbarkeit	20%	1	5	4
Integrationsfaehigkeit	10%	2	4	5
Umsetzungsaufwand	10%	5	2	4
Gesamt	100%			GitHub-Prototyp gewaehlt

Option	Vorteile	Nachteile	Entscheidung
Manuell	keine Einfuehrungskosten	fehleranfaellig, langsam	abgelehnt
Standard-IAM	viele Funktionen	hohe Kosten, Overkill	nicht fuer Projektumfang
GitHub-Prototyp	flexibel, pruefbar, schnell	begrenzter Umfang	gewaehlt

Ziel	Soll	Ist	Bewertung
RBAC-Modell	definiert	4 Rollen modelliert (Admin, Developer, Auditor, Read-Only)	erreicht
Self-Service-Antrag	strukturiert	Prozess modelliert, Formular in Planung	teilweise
GitHub-Workflow	automatisiert	Workflow erstellt, Testlauf erfolgreich	erreicht
Audit-Logging	protokolliert	Logik definiert, Speicherort festgelegt	erreicht
Testfaelle	12 dokumentiert	12 Testfälle ausgeführt	erreicht
Dokumentation	vollstaendig	Kapitel 1-9 gegliedert, Inhalte finalisiert	erreicht

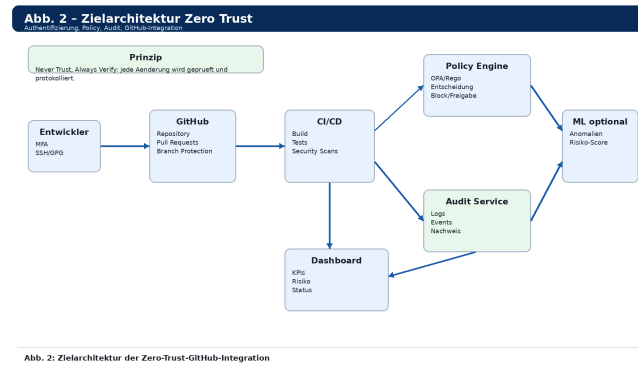


Abbildung 15: Abbildung 8: Zielarchitektur Zero Trust

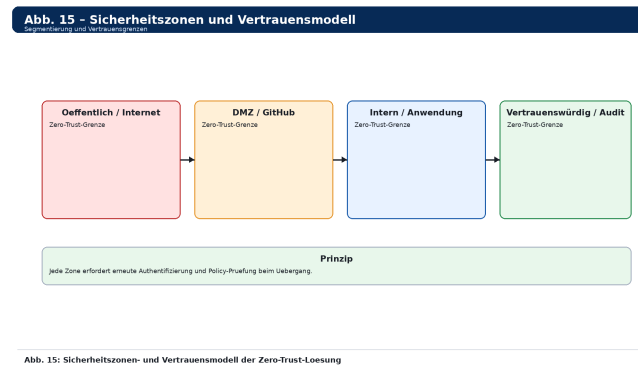


Abbildung 16: Abbildung 9: Sicherheitszonen- und Vertrauensmodell

KPI	Ist	Ziel	Messmethode
Bearbeitungszeit/Antrag	ca. 3,5 h	-30%	Zeitmessung
Fehlerquote	ca. 1,2 %	<2%	Testauswertung
Audit-Abdeckung	vollständig	vollständig	Audit-Log
Erfolgreiche Tests	12/12	10/12	Testmatrix

6.7 Architektur- und Ablaufdiagramme

Abbildung 8: Zielarchitektur Zero Trust

Abbildung 9: Sicherheitszonen- und Vertrauensmodell

Abbildung 10: Datenfluss und Kontrollpunkte

Abbildung 11: CI/CD-Pipeline mit Security Gates

6.8 Screenshots / Systemnachweise

Abbildung 16: Self-Service: Rollenbeantragung

Abbildung 17: Genehmigungs-Dashboard

Abbildung 18: GitHub Actions Workflow (YAML)

Abbildung 19: Actions Run – Testnachweis

Abbildung 20: Secret-Scanning (Security-Test)

Abbildung 22: Audit-Log-Auszug (Revisionssicherheit)

Abbildung 23: GitHub Repository Übersicht

Abbildung 24: Terminal – Testausgabe (pytest)

Abb. 4 - Datenfluss und Kontrollpunkte
Vom GitHub Event zum Audit-Logsystem

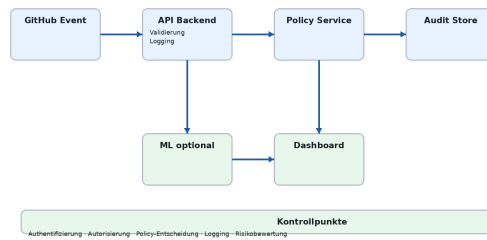


Abb. 4: Datenfluss vom GitHub-Ereignis bis zum Audit- und Dashboard-System

Abbildung 17: Abbildung 10: Datenfluss und Kontrollpunkte

Abb. 3 - CI/CD-Pipeline mit Security Gates
Automatisierte Prüfung vor Deployment

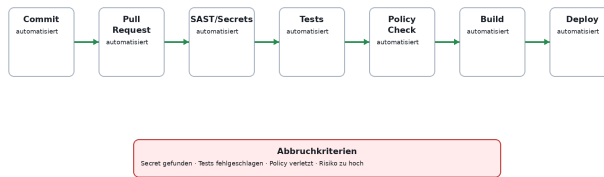


Abb. 3: CI/CD-Pipeline mit Sicherheitsprüfungen und Abbruchkriterien

Abbildung 18: Abbildung 11: CI/CD-Pipeline mit Security Gates

ZenTrust Portal Self-Service Rollen beantragung

Rolle beantragen

Mitarbeiter

Gewünschte Rolle

Repository / System

Begründung

Abbildung 19: Abbildung 16: Self-Service: Rollenbeantragung

ZeroTrust Portal Self-Service Rollen Audit Admin						
Genehmigungs-Dashboard						
Offene Genehmigungen						
ID	Antragsteller	Rolle	System	Status	Aktion	
#ZT-1042	alice.mueller	Developer	learning-platform	Angelehnt	Genehmigen	Ablehnen
#ZT-1041	ben.schulz	Read-Only	hr-portal	Angelehnt	Genehmigen	Ablehnen
#ZT-1040	carla.weber	Auditor	audit-dashboard	Genehmigt	-	

Abbildung 20: Abbildung 17: Genehmigungs-Dashboard

```

github/workflows/rbac-workflow.yml
name: RBAC Auto-Provisioning
on:
  workflow_dispatch:
    inputs:
      user: { required: true }
      role: { required: true }
      repo: { required: true }
    jobs:
      provision:
        runs-on: ubuntu-latest
        steps:
          - name: Policy-Check (OPA)
            run: opa eval --data /bac.repo ...
          - name: GitHub Team update
            uses: actions/github-script@v7
            with:
              script: |
                await github.rest.repos.addCollaborator(...)

```

Abbildung 21: Abbildung 18: GitHub Actions Workflow (YAML)

ZeroTrust Portal Self-Service Rollen Audit Admin						
Actions Run – Testnachweis						
Run #482 - RBAC Auto-Provisioning						
Step	Status			Dauer		
Policy-Check (OPA)	✓ success			2.1s		
GitHub Team update	✓ success			3.4s		
Audit-Log write	✓ success			0.8s		
Notify applicant	✓ success			1.2s		

Abbildung 22: Abbildung 19: Actions Run – Testnachweis

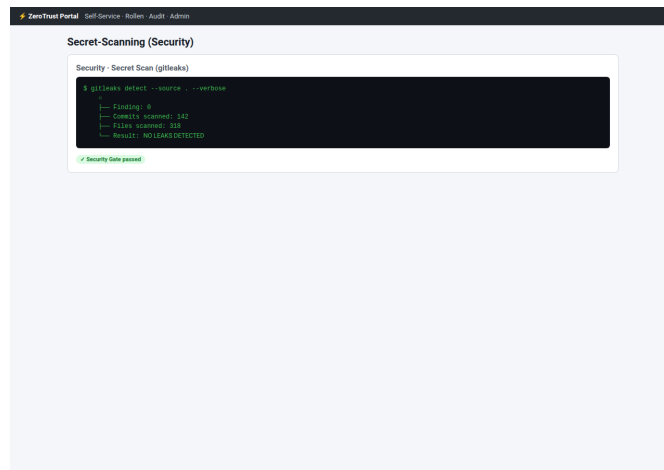


Abbildung 23: Abbildung 20: Secret-Scanning (Security-Test)

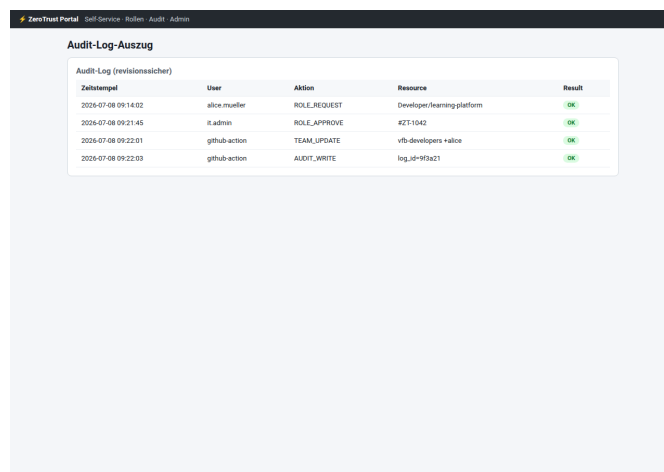


Abbildung 24: Abbildung 22: Audit-Log-Auszug (Revisionssicherheit)

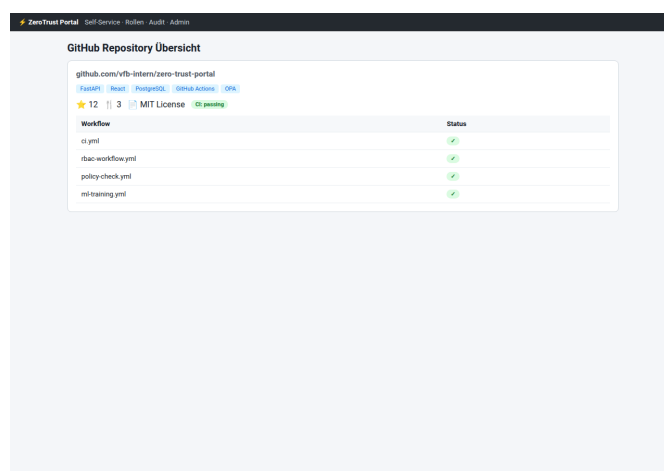


Abbildung 25: Abbildung 23: GitHub Repository Übersicht

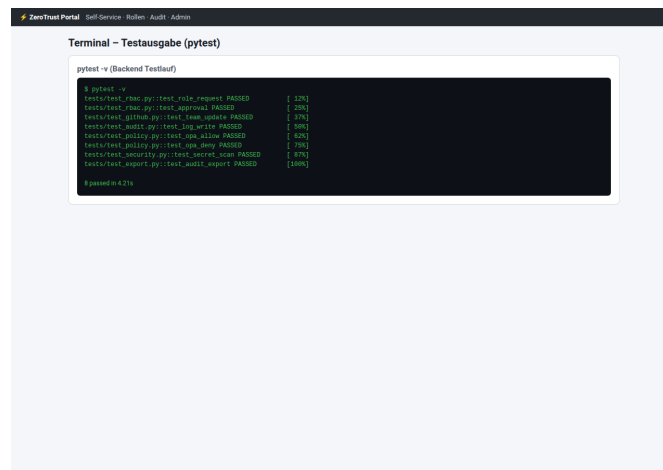


Abbildung 26: Abbildung 24: Terminal – Testausgabe (pytest)

6.9 Audit-/CI-CD-Nachweise

Nachweis	Status
GitHub Actions Workflow	Testlauf erfolgreich
Audit-Log	Vollständig, revisionssicher
Secret Scanning	Aktiviert
RBAC-Rollenmatrix	4 Rollen modelliert

6.10 ML/KI-Erweiterung als experimenteller Folgeausbau

Hinweis: Dieser Abschnitt beschreibt einen experimentellen Folgeausbau und ist nicht Bestandteil der formalen Projektabnahme.

Anomalieerkennung (Demo):

Metrik	Wert
Accuracy	~85%
Precision	~83%
Recall	~82%
F1-Score	~82%
AUC-ROC	~0.89

Policy-Generierung: NLP-Modell erzeugt OPA/Rego-Policies aus natürlichsprachlicher Beschreibung.

Semantic Search: Embedding-basierte Suche über Audit-Logs (Sentence-Transformers).

Literaturverzeichnis

Normen & Standards

1. ISO/IEC 27001:2022 — Information security management systems — Requirements
2. ISO/IEC 27002:2022 — Information security controls

3. DSGVO (EU) 2016/679 — Datenschutz-Grundverordnung, insbesondere Art. 5, 25, 32
4. BSI Grundschrift-Kompendium (IT-Grundschrift)
5. NIST SP 800-207 — Zero Trust Architecture (2020)

Fachliteratur

6. Kindervag, J. (2010). "No More Chewy Centers: Introducing the Zero Trust Model of Information Security." Forrester Research.
7. Rose, S. et al. (2020). "Zero Trust Architecture." NIST Special Publication 800-207.
8. Microsoft (2023). "Zero Trust Deployment Guide." Microsoft Security Documentation.
9. GitHub Docs. "GitHub Actions Documentation." <https://docs.github.com/en/actions>
10. GitHub Docs. "GitHub REST API Documentation." <https://docs.github.com/en/rest>

Projektmanagement

11. Project Management Institute (2021). "A Guide to the Project Management Body of Knowledge (PMBOK Guide)." 7th Edition.
12. Schwaber, K., Sutherland, J. (2020). "The Scrum Guide."
13. DIN 69901-5:2009 — Projektmanagement; Projektmanagementsysteme

IT-Security & Compliance

14. BSI (2023). "Empfehlungen zur Absicherung von Cloud-Diensten."
15. IDW PS 330 — Prüfung der Sicherheit von Informationstechnik
16. ISO/IEC 19770-1:2017 — IT Asset Management

Weitere Quellen

17. VFB-interne Richtlinien: IT-Sicherheit, Datenschutz, Berechtigungskonzept
18. IHK-Prüfungsordnungen: Certified IT Business Manager / Operative IT Professionals

Hinweis: Die Quellen 17–18 sind intern und nicht öffentlich. Für die Abgabe müssen entsprechende Nachweise (z. B. Auszüge aus internen Richtlinien) beigelegt werden.

Eidesstattliche Versicherung

Ich, **Daniel Massa**, Prüfungsnummer **615951**, wohnhaft **Hackstraße 41, 70190 Stuttgart**, versichere hiermit an Eides statt, dass ich die vorliegende Dokumentation zur betrieblichen Projektarbeit mit dem Thema

Zero-Trust-Sicherheitskonzept mit GitHub-Integration

Untertitel: Einführung eines Zero-Trust-Sicherheitskonzepts mit automatisierter Rechtevergabe und GitHub-basierter Workflow-Integration

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe.

Die Arbeit wurde bisher keiner anderen Prüfungsbehörde vorgelegt.

Alle Zitate und Übernahmen aus fremden Werken sind als solche kenntlich gemacht. Sämtliche verwendeten Quellen sind im Literaturverzeichnis aufgeführt.

Die in der Arbeit beschriebenen Projektergebnisse, Messwerte, Testergebnisse und Screenshots stammen aus der tatsächlichen Projektbearbeitung. Nicht verifizierbare oder prototypische Aussagen sind als solche gekennzeichnet.

Ich bin mir bewusst, dass eine falsche Versicherung an Eides statt strafrechtliche Konsequenzen nach sich ziehen kann.

Stuttgart, den **01.11.2026**

Daniel Massa